

Automating Go Test Coverage Using Goal-directed State Space Search Over Semantic Execution Paths, Using Coverage As A Constraint Oracle

Task

Home Assignment 1

Writing unit tests is time-consuming but critical for maintaining high-quality code. Design an automated system that continuously improves test coverage for Go repositories with minimal human intervention. The system should analyze a codebase to identify coverage gaps, generate meaningful and idiomatic Go unit tests for uncovered logic using an LLM, validate the generated tests by running them to ensure they compile, pass, and increase coverage, and then iteratively repeat this process until a configurable coverage threshold is reached or further gains become marginal. The application should accept a local repository path and a desired coverage percentage as inputs and run fully autonomously it should plan what to test, write tests, execute them, and iterate without manual steps.

For evaluation, use the sample Go repository <https://github.com/montanaflynn/stats> (delete all existing `_test.go` files before generating tests). You may use tools such as Codex, Claude Code, Cursor Agent, or Gemini CLI as agents or nodes within your system to generate and run tests instead of building a DIY agent/tooling stack, but the overall application must remain end-to-end autonomous.

Part-1: Understanding Go Testing

Ref: <https://go.dev/blog/cover>

1. How go test works

When we run `go test`, the tool performs a specialized build-and-run cycle:

- **Discovery:** Go looks for any files ending in `_test.go` in the current directory or specified packages. It ignores these files during a normal `go build`.
- **Compilation:** The tool creates a temporary "main" package that imports the code and the `testing` package. It identifies all functions starting with `TestXxx(t *testing.T)`, `BenchmarkXxx(b *testing.B)`, or `FuzzXxx(f *testing.F)`.
- **Execution:** It compiles this temporary package into a **test binary**, runs it, and then deletes the binary once the tests finish.

2. How coverage works

To get the coverage, we add the `-cover` flag to the `go test` command.

```
go test -cover
```

Go uses **instrumentation** to determine coverage. When we add the `-cover` flag, Go rewrites the source code before compilation.

For example, the code

```
func Size(a int) string {  
    if a < 10 {  
        return "small"  
    }  
    return "big"  
}
```

Is instrumented by adding counters at blocks for execution paths that can occur during the run.

```
func Size(a int) string {  
    GoCover.Count[0] = 1 // Track that the function started  
    if a < 10 {  
        GoCover.Count[2] = 1 // Track that the "if" block ran  
        return "small"  
    }  
    GoCover.Count[1] = 1 // Track that the "else/fallthrough" ran  
    return "big"  
}
```

These counters are then aggregated to determine the coverage of the source code.

3. Coverage Modes

The details of the instrumentation can be controlled using the `-covermode` flag. This flag can be set using the `-covermode` flag.

```
go test -cover -covermode=set
```

Go provides three cover modes for determining how coverage is measured.

Mode	Description
set (default)	Did each statement run? (Boolean: 0 or 1)
count	How many times did each statement run?
atomic	Same as count, but uses atomic operations (safe for multi-threaded/parallel tests).

4. Saving coverage report

To save the coverage data we use the `-coverprofile=<filename>` argument. This saves the calculated coverage metadata to the file specified by `<filename>` .

```
go test -cover -covermode=set -coverprofile=coverage.out
```

This outputs the coverage metadata to the `coverage.out` file, which can then be used for further analysis.

The coverage file looks something like this:

```
mode: set
github.com/montanaflynn/stats/correlation.go:8.61,13.24 3 0
github.com/montanaflynn/stats/correlation.go:13.24,15.3 1 0
github.com/montanaflynn/stats/correlation.go:17.2,17.14 1 0
github.com/montanaflynn/stats/correlation.go:17.14,19.3 1 0
github.com/montanaflynn/stats/correlation.go:21.2,24.30 3 0
github.com/montanaflynn/stats/correlation.go:24.30,26.3 1 0
github.com/montanaflynn/stats/correlation.go:28.2,29.36 2 1
```

The output is formatted as follows:

1. The first line `mode: set` : the mode used for calculating the coverage
2. Each of the following lines indicate a block in the code and are formatted as follow:

```
<module-name>/<file-path>:<block-start>,<block-end> <num-statements> <set/count>
```

We can take one line from above as an example

```
github.com/montanaflynn/stats/correlation.go:28.2,29.36 2 1
```

Part	Meaning
github.com/montanaflynn/stats	The package name
correlation.go	The file being measured
28.2,29.36	The start (line 28, col 2) and end (line 29, col 36) of the code block.
2	The number of statements within that block.
1	If the mode was set/atomic this is either 1 or 0 based on whether the block executed or not. If the mode was count this is the hit count of how many times the block was executed.

4. Analyzing the coverage report

The `go tool cover` allows us to parse the coverage data into a more readable and presentable format. It operates on the profile file (i.e. the `coverage.out`) generated during the test run. It provides two ways of analyzing the coverage report.

A. Function Breakdown (`-func`)

This view provides a text-based output that has aggregated output showing every function in the package and how well it was tested. This can be generated using the `-func` argument with the `go tool cover` command.

```
go tool cover -func=coverage.out
```

It provides an output that looks something like this

```
github.com/montanaflynn/stats/correlation.go:8:      Correlation      100.0%
github.com/montanaflynn/stats/correlation.go:33:     Pearson          0.0%
github.com/montanaflynn/stats/correlation.go:38:     AutoCorrelation  100.0%
github.com/montanaflynn/stats/cumulative_sum.go:4:   CumulativeSum    0.0%
github.com/montanaflynn/stats/data.go:7:            Get            0.0%
github.com/montanaflynn/stats/data.go:10:           Len            0.0%
.
.
.
github.com/montanaflynn/stats/util.go:36:           sortedCopyDif    0.0%
github.com/montanaflynn/stats/variance.go:6:        _variance        0.0%
github.com/montanaflynn/stats/variance.go:26:       Variance         0.0%
github.com/montanaflynn/stats/variance.go:31:       PopulationVariance 0.0%
github.com/montanaflynn/stats/variance.go:42:       SampleVariance  0.0%
github.com/montanaflynn/stats/variance.go:53:       Covariance       0.0%
github.com/montanaflynn/stats/variance.go:81:       CovariancePopulation 0.0%
total:                                           (statements)  28.1%
```

B. The HTML Visualizer (`-html`)

It generates a temporary HTML file and opens it in the default browser. It shows the source code and color codes the lines to indicate whether its covered, not covered, or not executable.

5. Using coverage information to make guided decisions

The coverage information from running `go test` can be used to find out which blocks of code need more testing. We can use the knowledge from uncovered blocks of code to guide what tests to write and achieve the maximum coverage possible. We will use this information to design an agent that will write tests for uncovered blocks of code in Go repositories.

Part-2 : Problem Definition

1. Test coverage as a state space exploration problem

A program's behavior is determined by factors like input values, internal variables, heap objects, global state, external state, and more. At any moment, a program is in a **state**.

```
Different Inputs -> Different States -> Different Execution Paths
```

To **achieve maximum coverage** for a given program, we need to make sure all the different execution paths are covered, i.e. we need to make sure we test the program in all possible **states**.

Therefore, the problem of ***achieving maximum coverage*** can also be framed as ***driving the program execution into as many distinct behavior (states) as possible**.

Which means, we can now frame the problem as a **state space search** of covering all the possible states a program can be in.

For example, the code

```
if x < 0 { // line 9
} else { // line 11

}
```

Splits the state space into two:

1. State where `x < 0`
2. State where `x >= 0`

Now, when `go test` reports

```
file.go:9.22,11.3 1 1
file.go:11.25,13.3 1 0
```

It means that the tests never reached an execution path where the `else` condition was executed, i.e. the state `x >= 0` was never tested.

A naive test generation will give us:

1. Random inputs
2. Repeated execution of same paths
3. Coverage Plateaus
4. Retries with no progress

Its like walking in a maze without remembering where you've been.

Whereas, state space exploration will give us:

1. Memory
 - which states have been reached
 - which execution paths are still unexplored
2. Direction
 - which constraints must be satisfied
 - which conditions are currently false
3. Termination
 - all reachable states explored
 - remaining states are unreachable or infeasible
 - marginal increase in coverage

With this in mind, we now define the problem formally.

2. Formal Problem Definition

A. Problem Framing

We model the **test generation for coverage completion** as a **search problem over program states** where:

- **State** = *(program + tests + coverage artifacts)*
- **Goal** = *(all target coverage obligations satisfied)*
- **Actions** = *(generate / modify tests)*
- **Cost** = *(complexity, flakiness risk, execution time, semantic distortion)*

This is **counterexample-guided state space exploration**, where coverage gaps act as *unsatisfied constraints*.

B. State Definition

Let a state be a tuple:

$$S = \langle P, T, C, U \rangle$$

Where:

- **P**: Program source code (fixed, immutable)
- **T**: Current test suite
- **C**: Coverage data derived from `coverage.out`
- **U**: Uncovered **coverage obligations** (derived set)

coverage obligations

Obligations are discovered from `coverage.out` by using the `mode=set`. This mode sets a value (=1) if the block executed, otherwise the value for the block is set to 0.

Each obligation is a tuple:

$$o = \langle f, r, k, \text{covered} \rangle$$

- `f` : source file

- `r` : source range (file:line.start–line.end)
- `k` : number of statements
- `covered` $\in \{0,1\}$

Let:

$$U = \{o \mid o.covered = 0\}$$

C. Initial State

The initial state is defined by S_0 where

$$S_0 = \langle P, T_0, C_0, U_0 \rangle$$

Where:

- T_0 = existing test suite
- C_0 = result of
`go test -coverprofile=coverage.out -covermode=set`
- U_0 = uncovered obligations

D. Goal State

A state S is **goal** if:

$$\forall o \in U_{\text{target}}, o.covered = 1$$

(where `U_target` may exclude unreachable or explicitly waived obligations)

OR

$$C_{\text{current}} \geq C_{\text{target}}$$

Where C_{current} and C_{target} are the current and target coverage.

E. Search Strategy

Strategy : **Hierarchical best-first search**

1. Select file `f` with **most coverage potential**
2. Select obligations $o \in U_f$
3. Generate **minimal tests** targeting only o
4. Evaluate
5. Iterate

The criteria **most coverage potential** means - files with least coverage and most uncovered blocks

F. Evaluation (test correctness)

After each transition, a test is accepted iff:

1. All tests execute
2. All tests pass
3. All uncovered blocks are covered

G. Rewrite Loop (self correction)

If a test does not cover the intended obligation $o \in U'$

Then:

1. Diagnose failure reason:
 - wrong precondition
 - missing setup
 - incorrect assumption
2. Generate **counterexample explanation**
3. Apply RewriteTest

This forms a **CEGIS loop**

Hypothesis → Test → Coverage → Counterexample → Revision

H. Termination Conditions

The search terminates when:

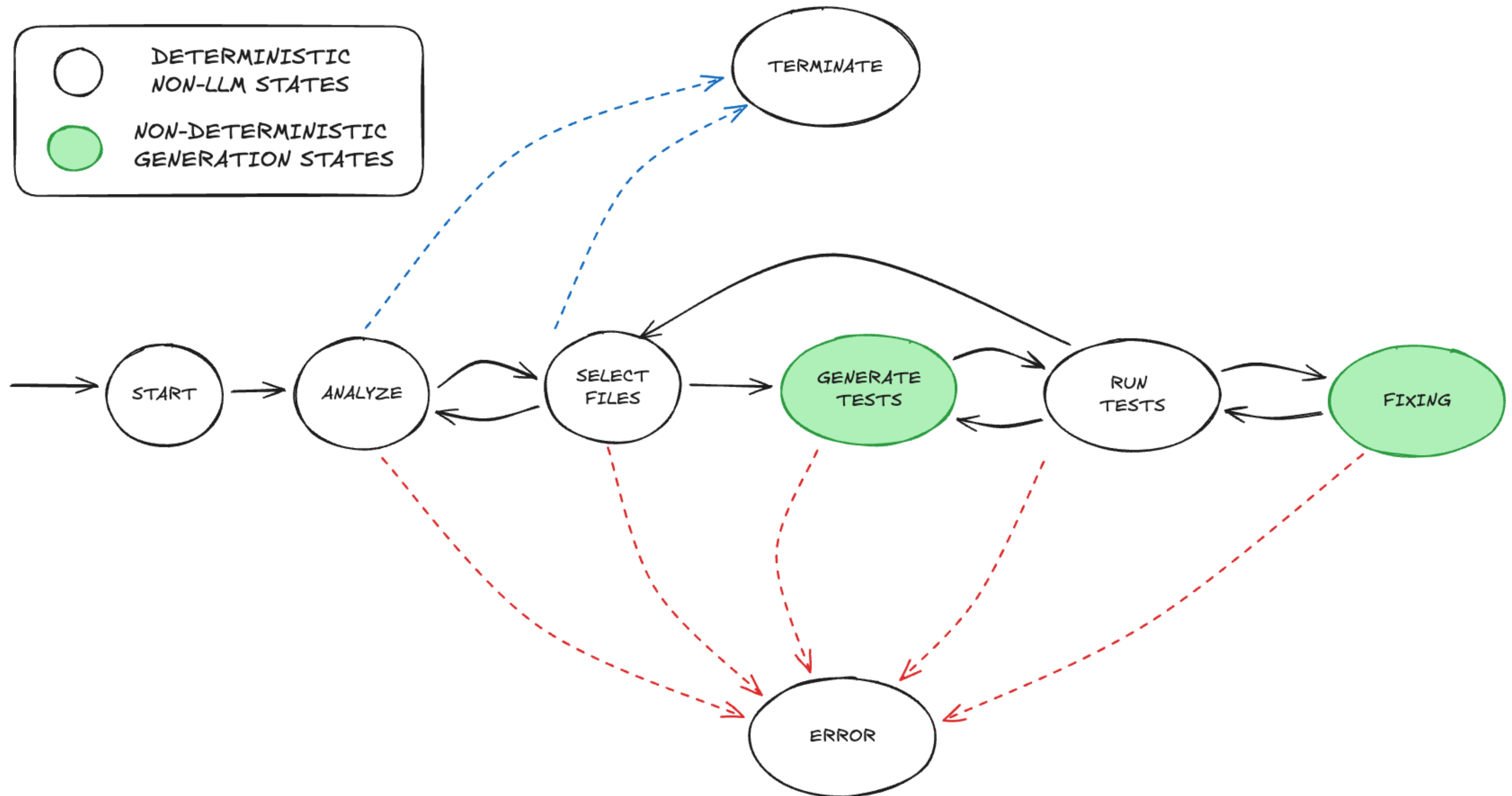
1. $U_{target} = \emptyset$
2. No valid actions remain for remaining obligations
3. marginal increase in coverage

Conclusion

This system is **not just writing tests**, it is performing **Goal-directed state space search over semantic execution paths, using coverage as a constraint oracle**

Part-3 : Conceptual Design

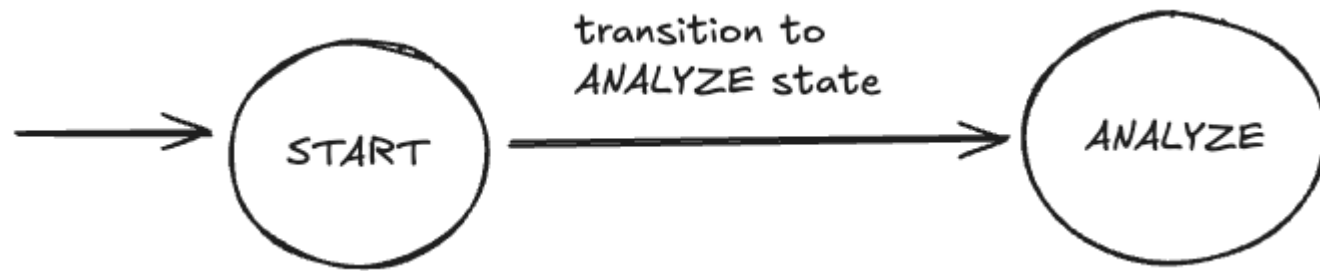
We design the agent as a **state space machine** that takes various actions based on its state.



The **Deterministic** states control the behavior of the program and guide the **LLMs** in the **Non-Deterministic** states to generate and fix tests.

1. START

The **START state** is to denote the start of the agent loop. It sets up the **ANALYZE STATE** and transitions itself to the **ANALYZE** state.

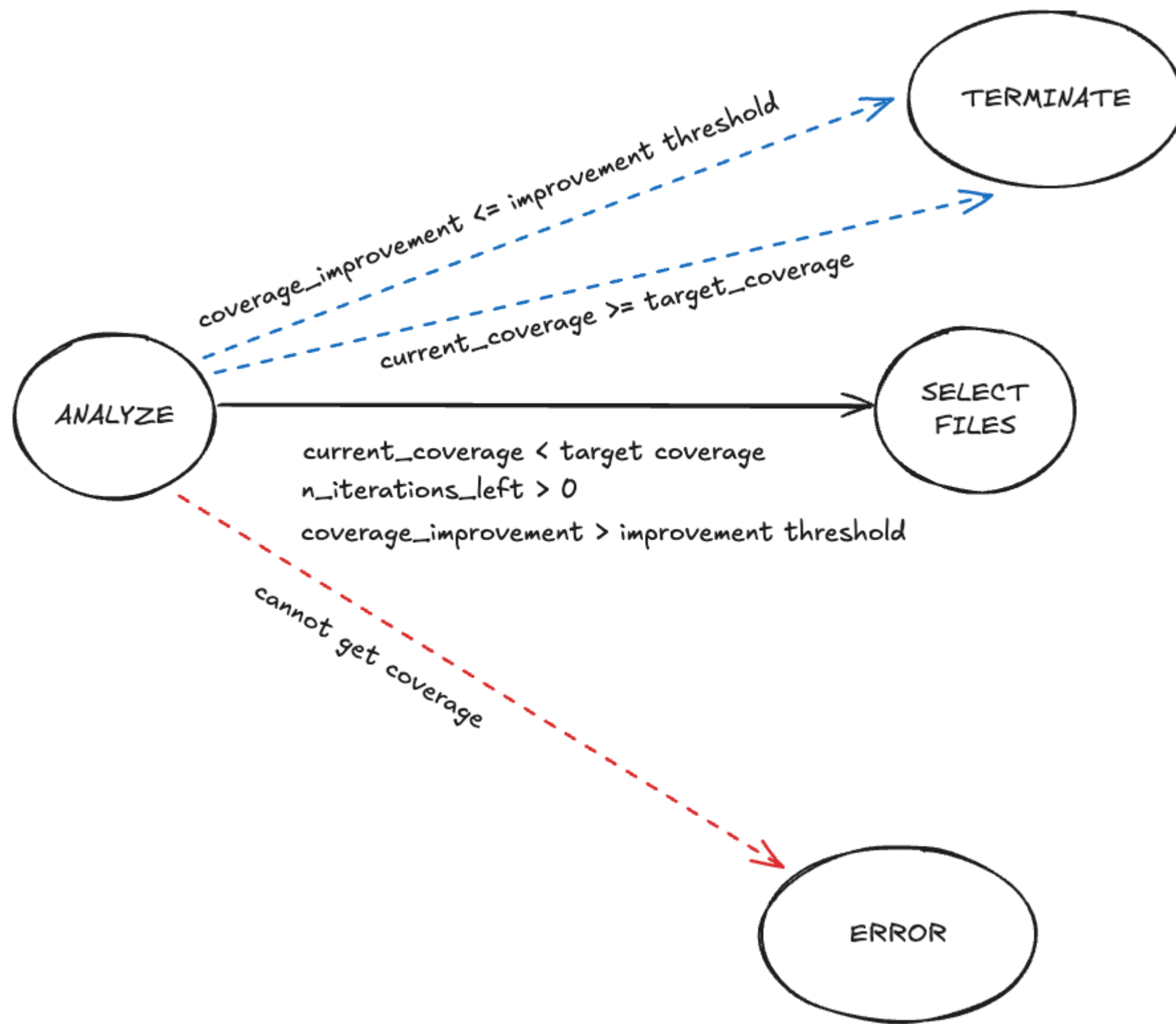


2. ANALYZE

The **ANALYZE** state is where the agent analyzes the following:

1. Is target coverage achieved?
2. Do we have more iterations left?
3. Are we only getting marginal coverage gains?
4. Should we continue?

Based on these factors, it makes a decision on which state to transition next.



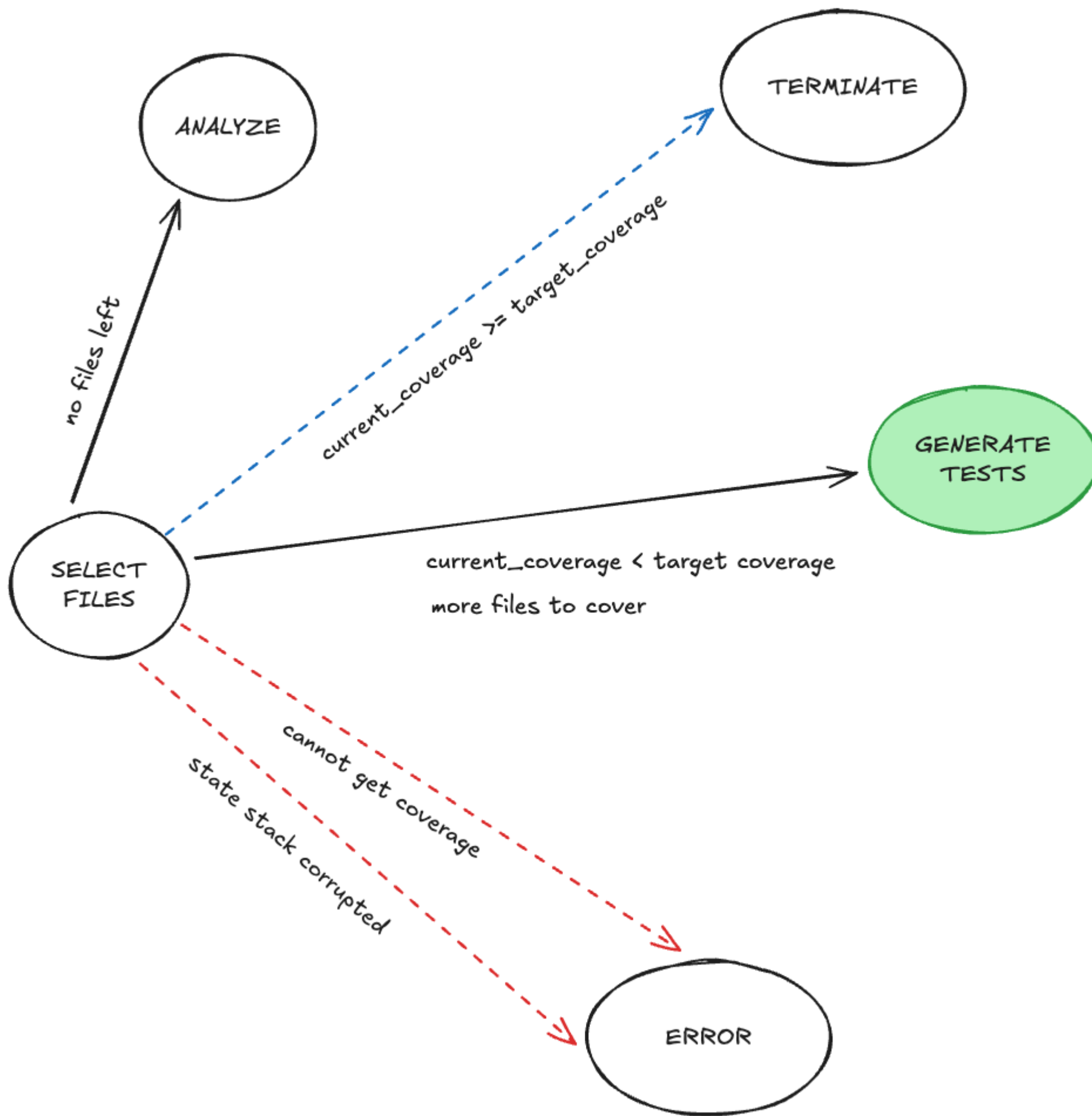
3. SELECT FILES

The **SELECT FILES** state analyzes the coverage report and takes a decision on which files to process next. If there are no files to process, it transitions back to the **ANALYZE** state.

This steps instructs the LLM to generate tests for a given file. It builds a system prompt that contains details on which file to write tests for and which specific blocks in the file to target. The input sent to the LLM is of the format:

```
Write tests for uncovered lines in File: /path/to/file.go
  Uncovered Blocks:
    - Start: 10.1, End: 15.5
    - Start: 30.2, End: 35.7
```

In addition to this, there is a **system prompt** which is mentioned in the appendix.



4. GENERATE TESTS

This part of execution is where the actual test generation happens. The LLM is provided with the **system prompt** and the **input prompt** and is tasked with writing tests for covering uncovered blocks.

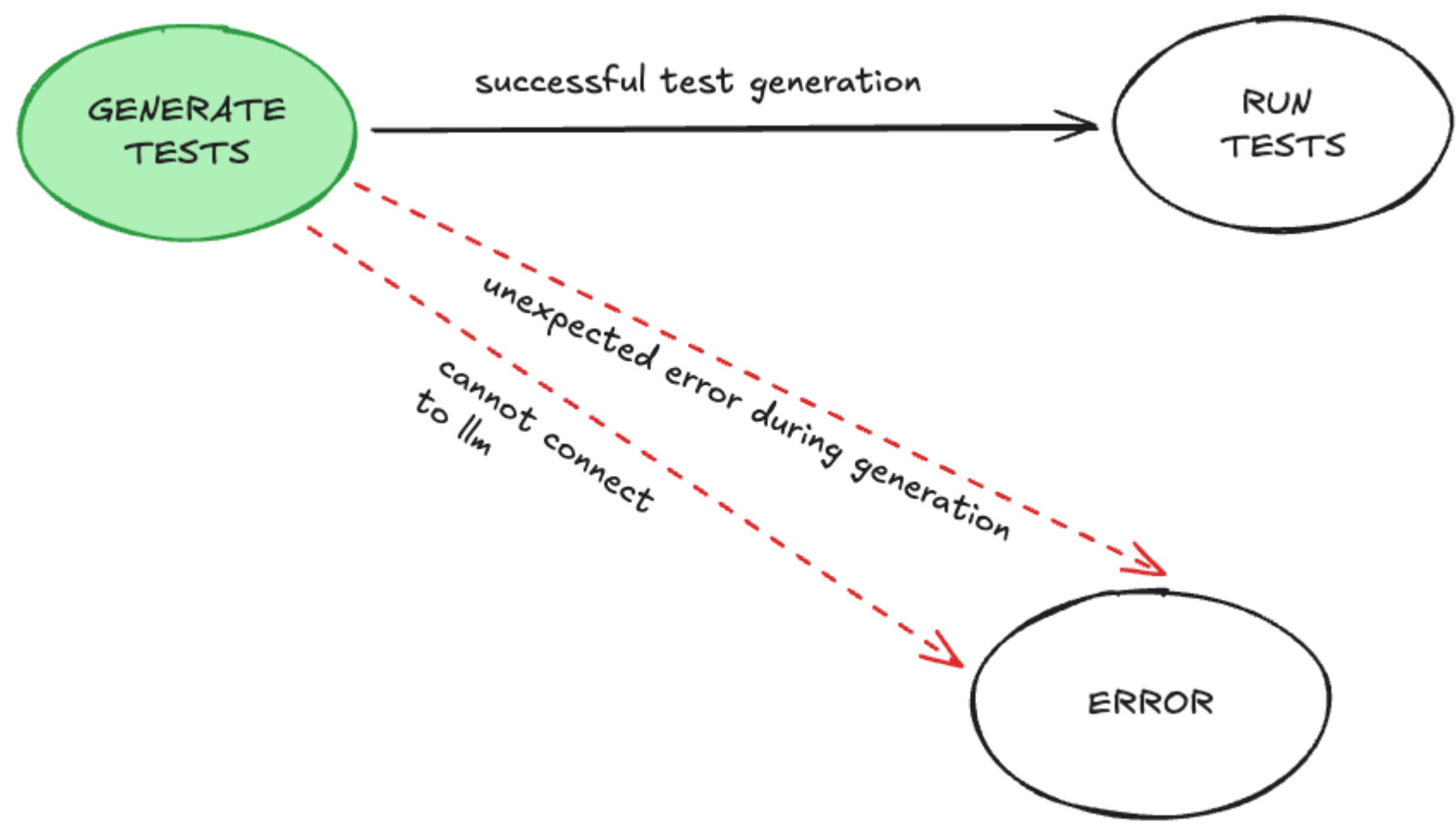
The input is of the format:

```
Write tests for uncovered lines in File: /path/to/file.go
Uncovered Blocks:
- Start: 10.1, End: 15.5
- Start: 30.2, End: 35.7
```

The LLM is provided with the following tools:

- 1. **Read** : To read contents of the files
- 2. **Write** : To write/create test files. This tool use is also intercepted at **pre tool use** to prevent the LLM from writing to source files. Only the access to `_test.go` files is allowed.
- 3. **Edit** : To edit test files. This tool use is also intercepted at **pre tool use** to prevent the LLM from writing to source files. Only the access to `_test.go` files is allowed.
- 4. **Glob** : To search the files present in the filesystem.
- 5. **Grep**: To read specific contents inside a file.

The access to **Bash** tool is denied, because we only let the deterministic components of the system to run tests and decide the correctness of tests.



5. RUN TESTS

This is the valuation state of the system where it evaluates whether the generated tests from the model truly satisfy the criteria.

A successful criteria is defined by:

- 1. All tests execute successfully
- 2. All known uncovered blocks are covered

If any of these are not satisfied, it transitions to the **FIXING** state.

If the tests did not execute correctly, it transitions to **FIXING** state with the input prompt:

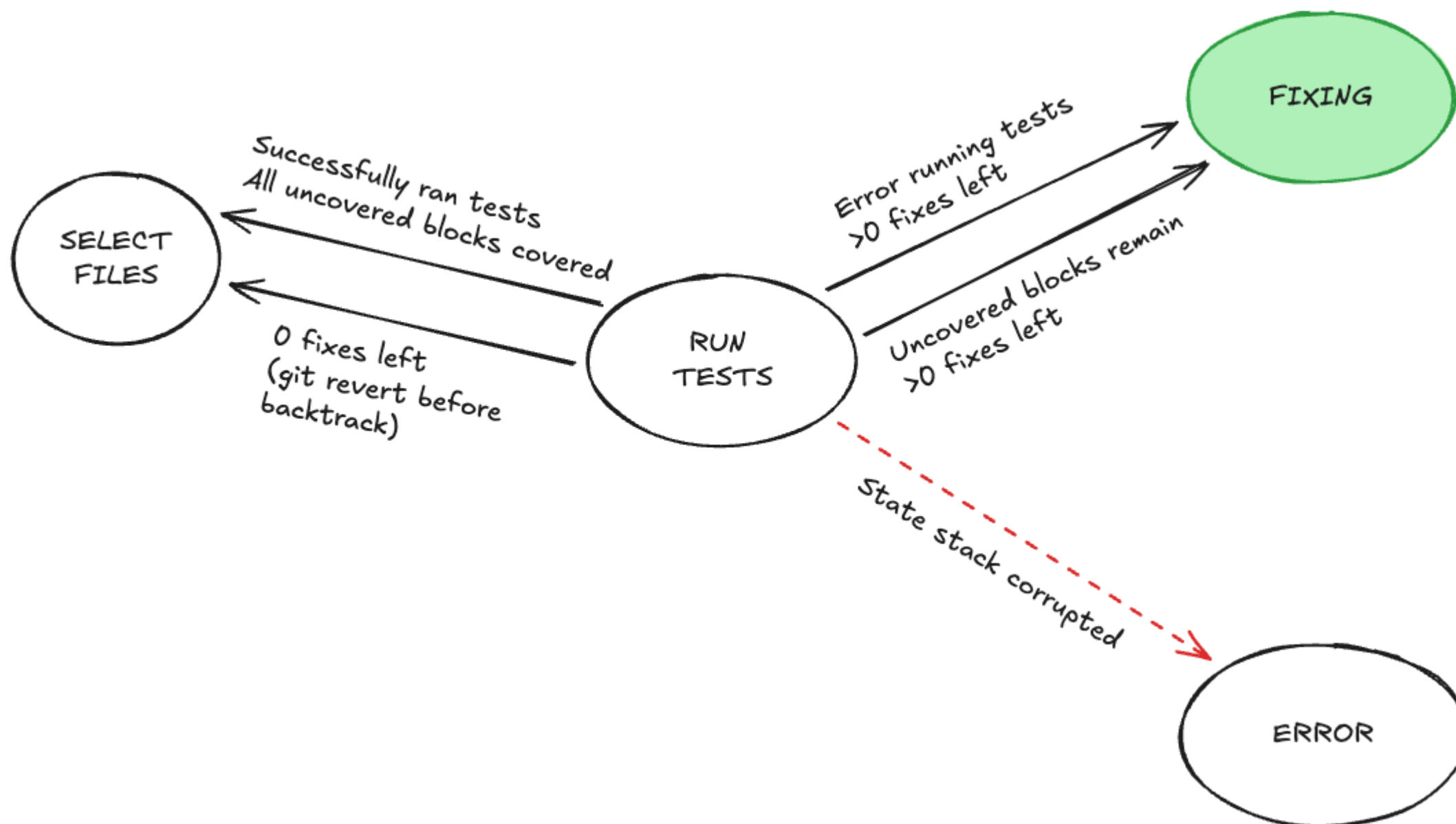
```
The tests you generated for File: /path/to/file.go have the following errors when run:  
[Error details here]  
Please fix the tests accordingly.
```

If any of the known uncovered blocks are not covered, it transitions to **FIXING** state with the input prompt:

```
After running the tests you generated for File: /path/to/file.go, the following uncovered blocks remain:  
Uncovered Blocks:  
- Start: 20.1, End: 25.3
```

It also maintains a count of how many times it asks the LLM to try fixes on a file. If the number of fixes left is 0, it reverts the changes and transitions back to the **SELECT FILES** state.

If all the criteria for successful test are qualified, it backtracks to the **SELECT FILES** state.



6. FIXING

This state is solely concerned with fixing the bugs produced in the Test Generation state. The fixing state deals with two kinds of requests:

- 1. Fixing wrong generated tests that do not compile
- 2. Covering the uncovered blocks from the Test Generation state.

The prompt for fixing wrong tests is:

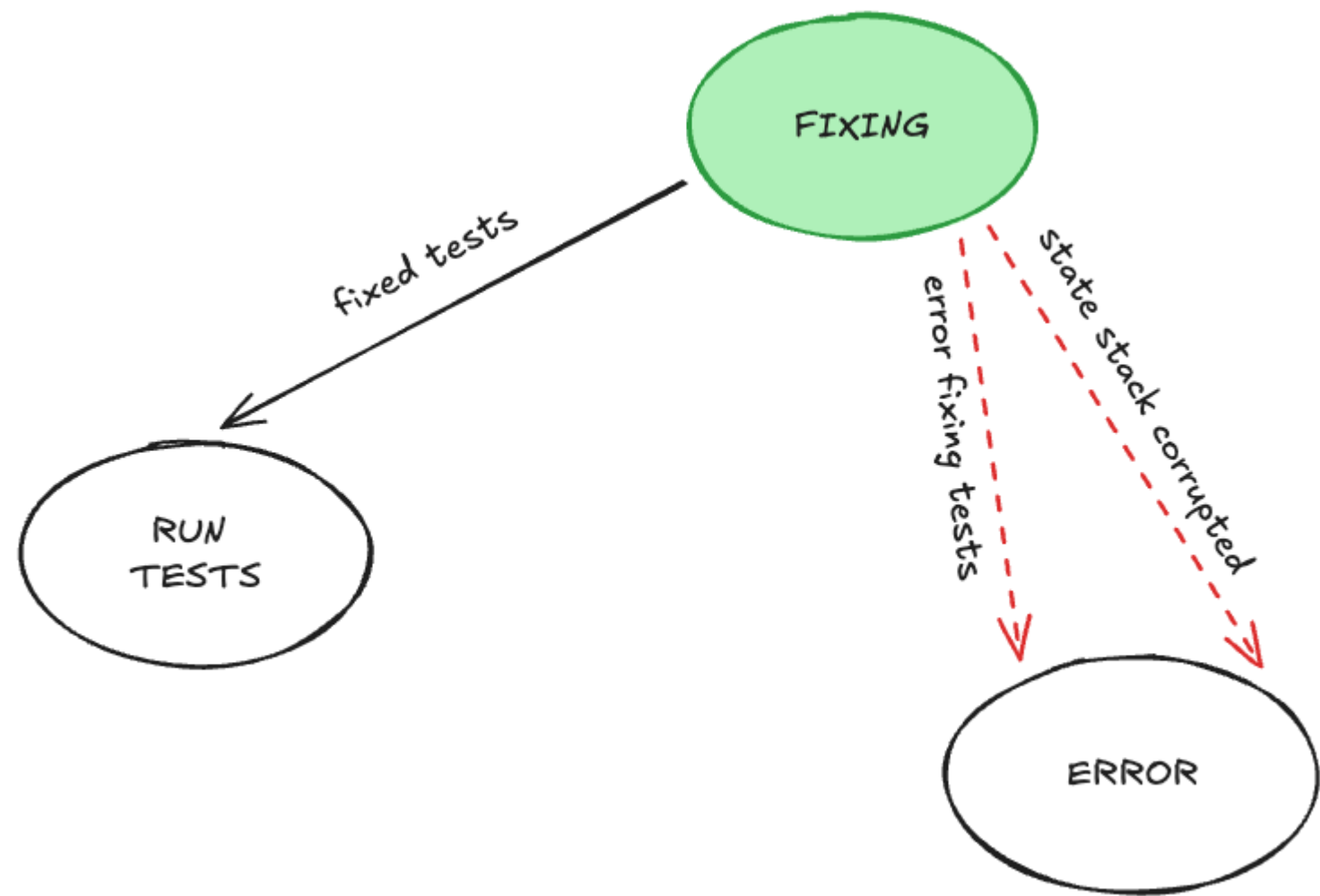
```
The tests you generated for File: /path/to/file.go have the following errors when run:  
[Error details here]  
Please fix the tests accordingly.
```

The prompt for fixing uncovered blocks is:

```
After running the tests you generated for File: /path/to/file.go, the following uncovered blocks remain:  
Uncovered Blocks:  
- Start: 20.1, End: 25.3
```

If the machine is in this state, it is passed the same SDK Client object used in the generation phase. This ensures that the context is preserved from the last call of Generate Tests.

Once the LLM is done generating, we backtrack to the last Run test state.



7. TERMINATE

This state is achieved when the agent has either achieved the goal or exhausted all the tries. Once this state is reached, the state control loop is terminated and no further state transition occurs.

8. ERROR

This state is achieved when the agent loop has encountered an unexpected error and we cannot transition to a valid state. Once this state is reached, the state control loop is terminated and no further state transition occurs.

Results

1. Achieving 0-10 (target)% coverage

In this phase we task the agent to get the test coverage to 10% in a repository which has 0% test coverage.

state transitions

#	STATE	COVERAGE %
1	START	
2	ANALYZE	0
3	SELECT_FILE	0
4	GENERATE_TEST	
5	RUN_TEST	
6	SELECT_FILE	15.2
7	TERMINATE	

token usage

Type	Usage
Input Tokens	22
Output Tokens	3730
Total Tokens	3752

total time taken - 47.73 seconds

coverage achieved - 15.2 %

Files updated:

- 1. `load_test.go`

2. Achieving 15.2% - 50 (target)% coverage

Here we build upon the previous phase and task the agent to achieve a **50%** test coverage in the repository that has **15.2%** test coverage.

state transitions

#	STATE	COVERAGE %
1	START	
2	ANALYZE	15.2
3	SELECT_FILE	15.2
4	GENERATE_TEST	
5	RUN_TEST	
6	SELECT_FILE	28.1
7	GENERATE_TEST	
8	RUN_TEST	
9	SELECT_FILE	35.1
10	GENERATE_TEST	
11	RUN_TEST	
12	SELECT_FILE	42.6
13	GENERATE_TEST	
14	RUN_TEST	
15	SELECT_FILE	75.4
16	TERMINATE	

token usage

Type	Usage
Input Tokens	4557
Output Tokens	15356
Total Tokens	19913

total time taken - 206 seconds

coverage achieved - 75.4 %

Files updated:

- 1. norm.go
- 2. regression.go

3. `variance.go`

4. `data.go`

3. Achieving 75.4% - 100 (target)% coverage

Here we build upon the previous phase and task the agent to achieve a **100%** test coverage in the repository that has **75.4%** test coverage.

state transitions

#	State	Cvg %	#	State	Cvg %	#	State	Cvg %	#	State	Cvg %
1	START		21	GENERATE_TEST		41	RUN_TEST		61	RUN_TEST	
2	ANALYZE	75.4	22	RUN_TEST		42	SELECT_FILE	97.0	62	SELECT_FILE	99.6
3	SELECT_FILE	75.4	23	FIXING		43	GENERATE_TEST		63	GENERATE_TEST	
4	GENERATE_TEST		24	RUN_TEST		44	RUN_TEST		64	RUN_TEST	
5	RUN_TEST		25	SELECT_FILE	93.0	45	SELECT_FILE	97.6	65	FIXING	
6	SELECT_FILE	80.7	26	GENERATE_TEST		46	GENERATE_TEST		66	RUN_TEST	
7	GENERATE_TEST		27	RUN_TEST		47	RUN_TEST		67	SELECT_FILE	99.6
8	RUN_TEST		28	SELECT_FILE		48	SELECT_FILE	97.9	68	GENERATE_TEST	
9	SELECT_FILE	85.1	29	GENERATE_TEST		49	GENERATE_TEST		69	RUN_TEST	
10	GENERATE_TEST		30	RUN_TEST		50	RUN_TEST		70	SELECT_FILE	99.7
11	RUN_TEST		31	SELECT_FILE	95.2	51	SELECT_FILE	98.4	71	GENERATE_TEST	
12	SELECT_FILE	88.2	32	GENERATE_TEST		52	GENERATE_TEST		72	RUN_TEST	
13	GENERATE_TEST		33	RUN_TEST		53	RUN_TEST		73	SELECT_FILE	99.7
14	RUN_TEST		34	SELECT_FILE	95.7	54	FIXING		74	GENERATE_TEST	
15	FIXING		35	GENERATE_TEST		55	RUN_TEST		75	RUN_TEST	
16	RUN_TEST		36	RUN_TEST		56	SELECT_FILE	99.1	76	SELECT_FILE	99.9
17	SELECT_FILE	90.1	37	FIXING		57	GENERATE_TEST		77	GENERATE_TEST	
18	GENERATE_TEST		38	RUN_TEST		58	RUN_TEST		78	RUN_TEST	
19	RUN_TEST		39	SELECT_FILE	96.3	59	SELECT_FILE	99.4	79	SELECT_FILE	100
20	SELECT_FILE	91.9	40	GENERATE_TEST		60	GENERATE_TEST		80	TERMINATE	100

token usage

Type	Usage
Input Tokens	405884
Output Tokens	458309
Total Tokens	864193

total time taken - 870 seconds
coverage achieved - 100 %

Files updated:

- 1. distances.go
- 2. describe.go
- 3. sample.go
- 4. geometric_distribution.go
- 5. round.go
- 6. percentile.go
- 7. mode.go
- 8. legacy.go
- 9. quartile.go
- 10. mean.go
- 11. util.go
- 12. correlation.go
- 13. deviation.go
- 14. entropy.go
- 15. outlier.go
- 16. errors.go
- 17. cumulative_sum.go
- 18. max.go
- 19. median.go
- 20. min.go
- 21. sigmoid.go
- 22. softmax.go

Appendix

System Prompt

You are a Go testing assistant specialized in achieving comprehensive test coverage.
Your task is to either write new tests or fix existing ones:

1. Context: User wants to improve test coverage for thier Go codebase.

Prompt: Write tests for uncovered lines in File: /path/to/file.go

Input Format:

Uncovered Blocks:

- Start: 10.1, End: 15.5
- Start: 30.2, End: 35.7

Explanation:

The Start and End indicate line.column positions of uncovered code blocks.

Task:

Based on the provided context and prompt, generate Go test functions that specifically target the uncovered lines

2. Context: User wants to fix errors in generated tests.

Prompt: The tests you generated for File: /path/to/file.go have the following errors when run:

[Error details here]

Please fix the tests accordingly.

Explanation:

You will be provided with error messages from running the tests you generated.

Task:

Based on the provided context and prompt, fix the errors.

3. Context: User tested the generated tests and found that some uncovered lines are still not covered.

Prompt: After running the tests you generated for File: /path/to/file.go, the following uncovered blocks remain:

Input Format:

Uncovered Blocks:

- Start: 20.1, End: 25.3

Explanation:

The Start and End indicate line.column positions of uncovered code blocks that remain uncovered after running the tests you generated.

Task:

Based on the provided context and prompt, generate additional Go test functions that specifically target the remaining uncovered lines.

Requirements:

1. Coverage Focus:

- Write tests ONLY for the specified uncovered lines
- Each test must execute the uncovered code paths
- Ensure all branches, error cases, and edge cases in uncovered blocks are tested

2. Test File Handling:

- If test file exists: append new tests to it
- If test file doesn't exist: create <original_filename>_test.go with correct package name
- Never modify existing test functions

3. Test Quality:

- Use table-driven tests where appropriate
- Test all conditional branches (if/else, switch cases)
- Test error returns and edge cases
- Use meaningful test function names: Test<FunctionName>_<Scenario>
- Include necessary setup and teardown
- Mock external dependencies if needed

4. Go Conventions:

- Import "testing" package
- Use t.Run() for subtests
- Use t.Error/t.Fatal appropriately
- Follow standard Go formatting

5. Output:

- Provide ONLY the Go test code
- No explanations, comments, or summaries
- No markdown formatting or code blocks
- Ready to write directly to test file

Prohibited Actions:

- Do not run go test
- Do not generate commands for running tests
- Do not generate coverage reports or HTML
- Do not execute any commands
- Do not provide analysis or explanations
- Do not modify covered code
- Do not write tests for already covered lines
- Do not provide summaries or commentary
- Do not provide output in any format other than raw Go code

Screenshots

0-10(target)% test coverage

EXPLORER

OPEN EDITORS

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

my-agent

11:28:41 PM

```
~/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent > main !3
time uv run main.py --repo-path /Users/rutu/stuff/vscode-workspaces/spectro/stats --target-coverage 10
warning: `VIRTUAL_ENV=/Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/my-agent/.venv` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
INFO:root:Starting Go Test Coverage Agent for repository: /Users/rutu/stuff/vscode-workspaces/spectro/stats, Target Coverage: 10.0%
INFO:root:[START] : Starting Go Test Coverage Improvement Agent
INFO:root:[START] Starting Go Test Coverage Improvement Agent | Repo: /Users/rutu/stuff/vscode-workspaces/spectro/stats | Target Coverage: 10.0%
INFO:root:[START] : Transitioning to new state: ANALYZE

Transitioned from START to ANALYZE
INFO:root:[ANALYZE] : Analyzing current test coverage
INFO:root:[ANALYZE] Analyzing current test coverage | Target Coverage: 10.0% | Current Coverage: 0.0% | Iterations Left: 4
INFO:root:[ANALYZE] : Transitioning to new state: SELECT_FILE

Transitioned from ANALYZE to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 10.0% | Current Coverage: 0.0% | Files Left: 27
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
Claude[load.go]: I'll read the load.go file to understand the code structure and write tests for the uncovered lines.
Claude[load.go]: Now let me check if a test file already exists:
INFO:root:PreToolUse (ID: toolu_01S4aeps3BjJPiJfP4STGs6a): Writing to test file: load_test.go - ALLOWED
[DONE] load.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: load.go
INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE
Selecting file for test generation
INFO:root:Target coverage 10.0% achieved with current coverage 15.2%. Terminating.
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 10.0% | Current Coverage: 15.2% | Files Left: 27
INFO:root:[SELECT_FILE] : Transitioning to new state: TERMINATE

Transitioned from SELECT_FILE to TERMINATE

=== AGENT TERMINATED ===
Final Coverage Achieved: 15.2%
Token Usage
Input Tokens: 22
Output Tokens: 3730
Total Tokens: 3752
```

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

15.2% - 50 (target)% coverage

EXPLORER

OPEN EDITORS

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

> OUTLINE

> TIMELINE

> GO

> PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

my-agent 11:42:33 PM

```
~/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent > main !3
time uv run main.py --repo-path /Users/rutu/stuff/vscode-workspaces/spectro/stats --target-coverage 50
warning: `VIRTUAL_ENV=/Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/my-agent/.venv` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
INFO:root:Starting Go Test Coverage Agent for repository: /Users/rutu/stuff/vscode-workspaces/spectro/stats, Target Coverage: 50.0%
INFO:root:[START] : Starting Go Test Coverage Improvement Agent
INFO:root:[START] Starting Go Test Coverage Improvement Agent | Repo: /Users/rutu/stuff/vscode-workspaces/spectro/stats | Target Coverage: 50.0%
INFO:root:[START] : Transitioning to new state: ANALYZE

Transitioned from START to ANALYZE
INFO:root:[ANALYZE] : Analyzing current test coverage
INFO:root:[ANALYZE] Analyzing current test coverage | Target Coverage: 50.0% | Current Coverage: 15.2% | Iterations Left: 4
INFO:root:[ANALYZE] : Transitioning to new state: SELECT_FILE

Transitioned from ANALYZE to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 50.0% | Current Coverage: 15.2% | Files Left: 27
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:root:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
Claude[norm.go]: I need to first read the norm.go file to understand the code structure and identify the uncovered lines.
Claude[norm.go]: Now I need to check if a test file already exists.
Claude[norm.go]: Now I'll create comprehensive tests for all the uncovered lines in norm.go:
INFO:root:PreToolUse (ID: toolu_01AfVVwLgKMjWjrrCHtz5znS): Writing to test file: norm_test.go - ALLOWED
[DONE] norm.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: norm.go
INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 50.0% | Current Coverage: 28.1% | Files Left: 26
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:root:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
Claude[regression.go]: I need to first read the regression.go file to understand the code structure and identify the uncovered lines.
Claude[regression.go]: Now I need to check if there's an existing test file and also look for the error definitions.
INFO:root:PreToolUse (ID: toolu_01VV68oFSZr6yRQECSvrWUmJ): Running allowed bash command: find /Users/rutu/stuff/vscode-workspaces/spectro/stats -name "*.go" -type f - ALLOWED
Claude[regression.go]: Now let me check the actual path of regression.go to ensure I write the test file in the correct location:
```

< main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[regression.go]: I need to first read the regression.go file to understand the code structure and identify the uncovered lines.

Claude[regression.go]: Now I need to check if there's an existing test file and also look for the error definitions.

INFO:root:PreToolUse (ID: toolu_01VV68oFSZr6yRQECSVrWUmJ): Running allowed bash command: find /Users/rutu/stuff/vscode-workspaces/spectro/stats -name "*.go" -type f - ALLOWED

Claude[regression.go]: Now let me check the actual path of regression.go to ensure I write the test file in the correct location:

INFO:root:PreToolUse (ID: toolu_015XVChReeDwP1yDjEuDKtD6): Running allowed bash command: pwd - ALLOWED

Claude[regression.go]: Good, I'm in the stats directory. Now I need to check for error variable names used in regression.go:

Claude[regression.go]: Now I'll create the test file with tests that cover all the uncovered blocks:

INFO:root:PreToolUse (ID: toolu_01Gzp6WdjBqZhGsPPnXQAGzJ): Writing to test file: /Users/rutu/stuff/vscode-workspaces/spectro/stats/regression_test.go - ALLOWED

[DONE] regression.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: regression.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 50.0% | Current Coverage: 35.1% | Files Left: 25

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[variance.go]: I'll read the variance.go file to understand the code structure and then write tests for the uncovered lines.

Claude[variance.go]: Now I'll check for any error types referenced in the code:

Claude[variance.go]: Now I'll write the test file for variance.go:

INFO:root:PreToolUse (ID: toolu_01JPB23t6999BfDTX2GRy7Xz2): Writing to test file: variance_test.go - ALLOWED

[DONE] variance.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: variance.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 50.0% | Current Coverage: 42.6% | Files Left: 24

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py M

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

> old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

env/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[variance.go]: I'll read the variance.go file to understand the code structure and then write tests for the uncovered lines.

Claude[variance.go]: Now I'll check for any error types referenced in the code:

Claude[variance.go]: Now I'll write the test file for variance.go:

INFO:root:PreToolUse (ID: toolu_01JPB23t6999BfDTX2GRy7Xz): Writing to test file: variance_test.go - ALLOWED

[DONE] variance.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: variance.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 50.0% | Current Coverage: 42.6% | Files Left: 24

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.v

env/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[data.go]: I'll read the data.go file to understand the code structure and generate tests for the uncovered lines.

Claude[data.go]: Now let me check if a test file already exists:

Claude[data.go]: Now I'll create the test file with tests targeting all uncovered lines:

INFO:root:PreToolUse (ID: toolu_01MQ9AyuCQUUamHskn2yuZ6r): Writing to test file: data_test.go - ALLOWED

[DONE] data.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: data.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:Target coverage 50.0% achieved with current coverage 75.4%. Terminating.

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 50.0% | Current Coverage: 75.4% | Files Left: 24

INFO:root:[SELECT_FILE] : Transitioning to new state: TERMINATE

Transitioned from SELECT_FILE to TERMINATE

=== AGENT TERMINATED ===

Final Coverage Achieved: 75.4%

Token Usage

Input Tokens: 4557

Output Tokens: 15356

Total Tokens: 19913

< main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ } Python

3.14.0 (my-agent)

Dependi

75.4% - 100 (target)% coverage

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

my-agent 12:05:02 AM

```
~/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent > main !3
time uv run main.py --repo-path /Users/rutu/stuff/vscode-workspaces/spectro/stats --target-coverage 100
warning: `VIRTUAL_ENV=/Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/my-agent/.venv` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
INFO:root:Starting Go Test Coverage Agent for repository: /Users/rutu/stuff/vscode-workspaces/spectro/stats, Target Coverage: 100.0%
INFO:root:[START] : Starting Go Test Coverage Improvement Agent
INFO:root:[START] Starting Go Test Coverage Improvement Agent | Repo: /Users/rutu/stuff/vscode-workspaces/spectro/stats | Target Coverage: 100.0%
INFO:root:[START] : Transitioning to new state: ANALYZE

Transitioned from START to ANALYZE
INFO:root:[ANALYZE] : Analyzing current test coverage
INFO:root:[ANALYZE] Analyzing current test coverage | Target Coverage: 100.0% | Current Coverage: 75.4% | Iterations Left: 4
INFO:root:[ANALYZE] : Transitioning to new state: SELECT_FILE

Transitioned from ANALYZE to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 75.4% | Files Left: 27
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
INFO:root:PreToolUse (ID: toolu_01FPkgYqdgP3A9rh4MfF1rh): Running allowed bash command: ls distances_test.go 2>/dev/null || echo "File does not exist" - ALLOWED
INFO:root:PreToolUse (ID: toolu_01U3HA7i5ZY2EdCxChQBvkhw): Writing to test file: distances_test.go - ALLOWED
[DONE] distances.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: distances.go
INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 80.7% | Files Left: 26
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
Claude[describe.go]: I'll read the file to understand the code structure and then write tests for the uncovered lines.
Claude[describe.go]: Now I'll check if a test file exists:
Claude[describe.go]: Now I need to understand the types and functions used in the code to write proper tests. Let me check the package structure:
Claude[describe.go]: Now I'll write the test file to cover all the uncovered lines:
INFO:root:PreToolUse (ID: toolu_01PeZiSVbuyNa1UCunM3paJx): Writing to test file: describe_test.go - ALLOWED
[DONE] describe.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: describe.go
```

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py M

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: describe.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 85.1% | Files Left: 25

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

INFO:root:PreToolUse (ID: toolu_011M98yFzAdnR17uDcm13vgA): Writing to test file: sample_test.go - ALLOWED

[DONE] sample.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: sample.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 88.2% | Files Left: 24

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[geometric_distribution.go]: I'll read the geometric_distribution.go file to understand the code structure and identify the uncovered lines.

Claude[geometric_distribution.go]: Now I'll check if a test file already exists:

Claude[geometric_distribution.go]: Now I'll check for the error definitions used in the code:

Claude[geometric_distribution.go]: Now I'll create the test file with tests covering all the uncovered lines:

INFO:root:PreToolUse (ID: toolu_01HnWjDVSly1t3LMuukwVRaM): Writing to test file: geometric_distribution_test.go - ALLOWED

[DONE] geometric_distribution.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: geometric_distribution.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

INFO:root:[RUN_TEST] Running tests for selected file | File: geometric_distribution.go | Fixes Left: 2

INFO:root:[RUN_TEST] : Transitioning to new state: FIXING

< >

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ } Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Transitioned from RUN_TEST to FIXING

INFO:root:[FIXING] : Fixing failing tests

INFO:root:Fixing errors for file geometric_distribution.go: 'Error running tests: --- FAIL: TestExpGeom_InvalidParameter (0.00s)\n --- FAIL: TestExpGeom_InvalidParameter/p_equals_0 (0.00s)\n\ngeometric_distribution_test.go:108: ExpGeom() error = <nil>, want Must not contain negative values.\n\ngeometric_distribution_test.go:111: ExpGeom() exp = +Inf, want NaN\nFAIL\ncoverage: 90.2% of statements\nexit status 1\nFAIL\tgithub.com/montanaflynn/stats\t0.532s\n'

[DONE Fix] geometric_distribution.go; success

Transitioned from FIXING to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 90.1% | Files Left: 23

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[round.go]: I'll read the round.go file to understand the code and then write tests for the uncovered lines.

INFO:root:PreToolUse (ID: toolu_01HXxBcw3y3VqKemS44z2dQ5): Running allowed bash command: test -f round_test.go && echo "exists" || echo "not exists" - ALLOWED

INFO:root:PreToolUse (ID: toolu_012PYkYTW7J6CyBHeKsnmsV2): Writing to test file: round_test.go - ALLOWED

[DONE] round.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: round.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 91.9% | Files Left: 22

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[percentile.go]: I'll read the percentile.go file to understand the code structure and identify the uncovered lines.

Claude[percentile.go]: Now let me check if a test file already exists:

Claude[percentile.go]: Let me check for any related types and errors that are referenced in the code:

Claude[percentile.go]: Now I'll create the test file with tests that cover all the uncovered lines:

INFO:root:PreToolUse (ID: toolu_014JgkbiuMXGVtoPVdxKEvmM): Writing to test file: /Users/rutu/stuff/vscode-workspaces/spectro/stats/percentile_test.go - ALLOWED

[DONE] percentile.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: percentile.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

> old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

INFO:root:[RUN_TEST] Running tests for selected file | File: percentile.go | Fixes Left: 2

INFO:root:[RUN_TEST] : Transitioning to new state: FIXING

Transitioned from RUN_TEST to FIXING

INFO:root:[FIXING] : Fixing failing tests

INFO:root:Fixing errors for file percentile.go: 'Error running tests: FAIL\tgithub.com/montanaflynn/stats [build failed]\n\n# github.com/montanaflynn/stats [github.com/montanaflynn/stats.test]\n./percentile_test.go:4:2: "math" imported and not used\n'

[DONE Fix] percentile.go; success

Transitioned from FIXING to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 93.0% | Files Left: 21

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[mode.go]: I'll read the file to understand the code structure and uncovered lines, then write tests for them.

Claude[mode.go]: Now I'll check the legacy.go file to see if EmptyInputErr is defined there:

Claude[mode.go]: Now I'll write the test file for mode.go:

INFO:root:PreToolUse (ID: toolu_01GoNFbuFsRCq9nobnGdm14j): Writing to test file: /Users/rutu/stuff/vscode-workspaces/spectro/stats/mode_test.go - ALLOWED

[DONE] mode.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: mode.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 94.2% | Files Left: 20

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[legacy.go]: I'll help you write tests for the uncovered lines in legacy.go. Let me first read the file to understand the code structure.

Claude[legacy.go]: Now let me check if a test file already exists:

Claude[legacy.go]: Now let me check the types and functions being called to understand what I need to test:

< main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

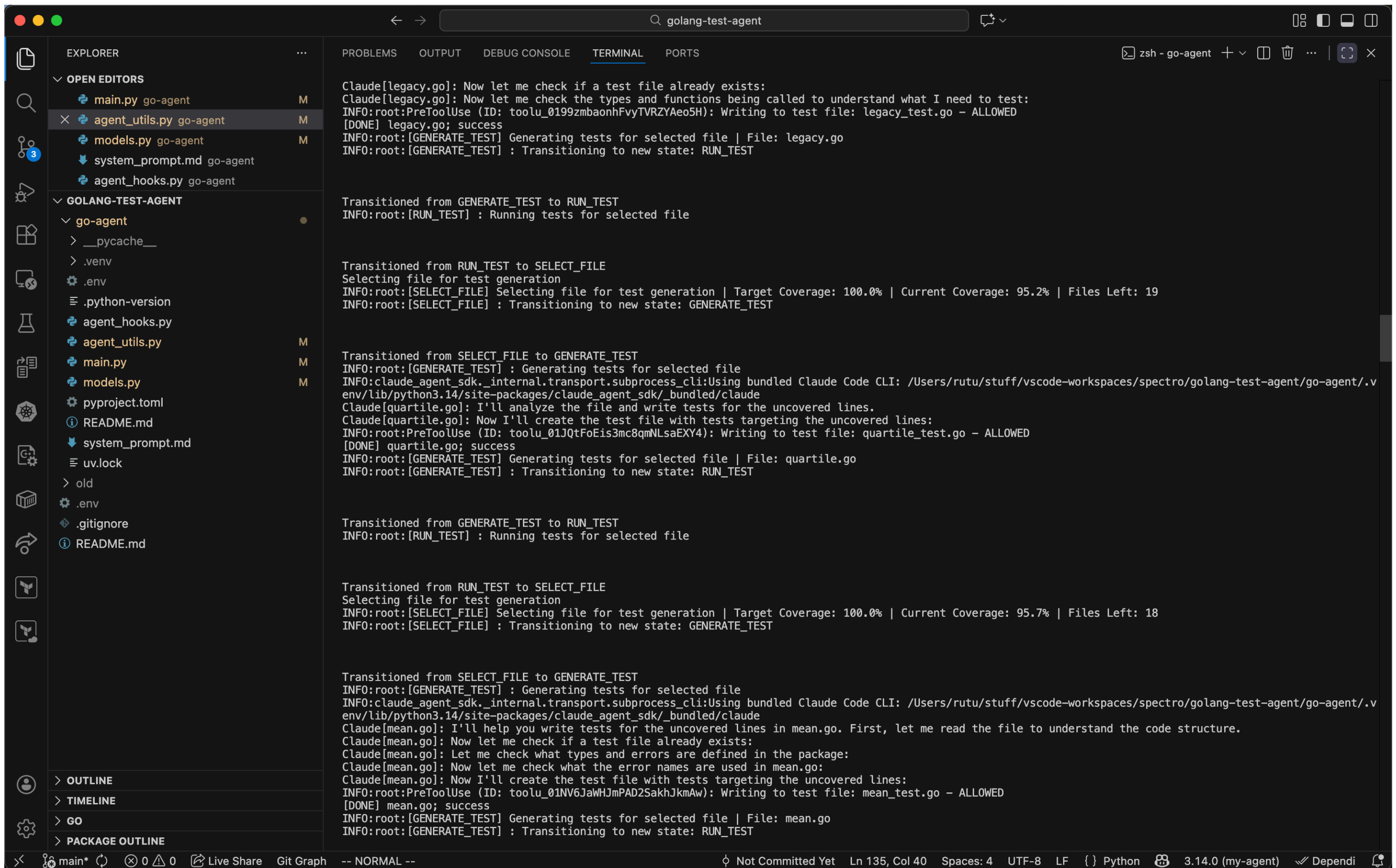
UTF-8

LF

{ } Python

3.14.0 (my-agent)

Dependi



golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

> old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

INFO:root:[RUN_TEST] Running tests for selected file | File: mean.go | Fixes Left: 2

INFO:root:[RUN_TEST] : Transitioning to new state: FIXING

Transitioned from RUN_TEST to FIXING

INFO:root:[FIXING] : Fixing failing tests

INFO:root:Fixing errors for file mean.go: 'Error running tests: FAIL\tgithub.com/montanaflynn/stats [build failed]\n\n# github.com/montanaflynn/stats [github.com/montanaflynn/stats.test]\n\n./mean_test.go:4:2: "math" imported and not used\n'

[DONE Fix] mean.go; success

Transitioned from FIXING to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 96.3% | Files Left: 17

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk_internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[util.go]: I'll help you write tests for the uncovered lines in util.go. Let me first read the file to understand the code structure.

Claude[util.go]: Now let me check if a test file already exists:

Claude[util.go]: Now let me check the Round function and Float64Data type to understand dependencies:

Claude[util.go]: Now I'll create the test file with tests targeting the uncovered lines:

INFO:root:PreToolUse (ID: toolu_01PZGP1N59CTAfthqopDjQTj): Writing to test file: util_test.go - ALLOWED

Claude[util.go]: package stats

import "testing"

func Test_float64ToInt(t *testing.T) {

tests := []struct {

name string

input float64

expect int

{

{

name: "positive float",

input: 5.7,

expect: 6,

},

{

name: "negative float",

input: -3.4,

expect: -3,

},

{

name: "zero",

input: 0.0,

expect: 0,

}

}

}

<>

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ } Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

```
        input: 0.0,
        expect: 0,
    },
    {
        name: "large float",
        input: 999.9,
        expect: 1000,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        result := float64ToInt(tt.input)
        if result != tt.expect {
            t.Errorf("float64ToInt(%v) = %v, want %v", tt.input, result, tt.expect)
        }
    })
}

}

func Test_sortedCopyDif_Unsorted(t *testing.T) {
    input := Float64Data{3.0, 1.0, 2.0}
    result := sortedCopyDif(input)

    if len(result) != 3 {
        t.Errorf("sortedCopyDif() length = %v, want 3", len(result))
    }

    if result[0] != 1.0 || result[1] != 2.0 || result[2] != 3.0 {
        t.Errorf("sortedCopyDif() = %v, want [1 2 3]", result)
    }

    if input[0] != 3.0 {
        t.Errorf("sortedCopyDif() modified input")
    }
}

[DONE] util.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: util.go
INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 97.0% | Files Left: 16
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
Claude[correlation.go]: I'll help you write tests for the uncovered lines in correlation.go. Let me first read the file to understand the code structure.
Claude[correlation.go]: Now let me check if a test file already exists:
```

< >

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

EXPLOER

...

OPEN EDITORS

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

+

⌵

🗑

⋮

🔍

✕

Claude[correlation.go]: Now let me check if a test file already exists:

Claude[correlation.go]: Let me check for the error types and dependent functions that I'll need to reference:

Claude[correlation.go]: Now let me check for the Float64Data type:

Claude[correlation.go]: Now I'll write the test file:

INFO:root:PreToolUse (ID: toolu_011QVQZYkdbdaR3DbxZLDUAg): Writing to test file: correlation_test.go - ALLOWED

[DONE] correlation.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: correlation.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 97.6% | Files Left: 15

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[deviation.go]: I'll read the deviation.go file to understand the code structure and identify the uncovered lines.

Claude[deviation.go]: Now let me check if a test file exists:

Claude[deviation.go]: Let me check what other files exist to understand the package structure better:

Claude[deviation.go]: Let me check one of the existing test files to understand the package structure and testing patterns:

Claude[deviation.go]: Now let me check the errors.go file to understand EmptyInputErr:

Claude[deviation.go]: Now I'll write the test file. The uncovered blocks are the empty input error checks in lines 12-14, 34-36, and 48-50. I need to create test s that trigger these error conditions:

INFO:root:PreToolUse (ID: toolu_01VYsAcMEPamNHfmNpwPSwcB): Writing to test file: deviation_test.go - ALLOWED

[DONE] deviation.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: deviation.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 97.9% | Files Left: 14

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[entropy.go]: I'll analyze the entropy.go file to understand the uncovered code blocks and write tests to cover them.

Claude[entropy.go]: Now I'll write the tests for the uncovered lines:

INFO:root:PreToolUse (ID: toolu_015Y1WeWzPqS36AHhL42kUz3): Writing to test file: entropy_test.go - ALLOWED

<

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Claude[entropy.go]: Now I'll write the tests for the uncovered lines:
INFO:root:PreToolUse (ID: toolu_015Y1WeWzPqS36AHhL42kUz3): Writing to test file: entropy_test.go - ALLOWED
[DONE] entropy.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: entropy.go
INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE
Selecting file for test generation
INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 98.4% | Files Left: 13
INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST
INFO:root:[GENERATE_TEST] : Generating tests for selected file
INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude
Claude[outlier.go]: I'll help you write tests for the uncovered lines in outlier.go. Let me first read the file to understand the code structure.
Claude[outlier.go]: Now let me check if a test file already exists:
Claude[outlier.go]: Now let me check for the Float64Data type and other dependencies:
Claude[outlier.go]: Now let me create the test file with tests targeting the uncovered lines:
INFO:root:PreToolUse (ID: toolu_01L5XoTbpYmvG2aWojSRFtZh): Writing to test file: /Users/rutu/stuff/vscode-workspaces/spectro/stats/outlier_test.go - ALLOWED
Claude[outlier.go]: package stats

import "testing"

func TestQuartileOutliers_EmptyInput(t *testing.T) {
_, err := QuartileOutliers(Float64Data{})
if err != EmptyInputErr {
t.Errorf("Expected EmptyInputErr for empty input, got %v", err)
}
}

func TestQuartileOutliers_ExtremeOutliers(t *testing.T) {
data := Float64Data{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, -100}
result, err := QuartileOutliers(data)
if err != nil {
t.Errorf("Expected no error, got %v", err)
}
if len(result.Extreme) == 0 {
t.Error("Expected extreme outliers to be detected")
}
}

func TestQuartileOutliers_MildOutliers(t *testing.T) {
data := Float64Data{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15}
result, err := QuartileOutliers(data)
if err != nil {
t.Errorf("Expected no error, got %v", err)
}
if len(result.Mild) == 0 {
t.Error("Expected mild outliers to be detected")
}
}

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

if len(result.Mild) == 0 {
 t.Error("Expected mild outliers to be detected")
}
}
[DONE] outlier.go; success
INFO:root:[GENERATE_TEST] Generating tests for selected file | File: outlier.go
INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file
INFO:root:[RUN_TEST] Running tests for selected file | File: outlier.go | Fixes Left: 2
INFO:root:[RUN_TEST] : Transitioning to new state: FIXING

Transitioned from RUN_TEST to FIXING
INFO:root:[FIXING] : Fixing failing tests
INFO:root:Fixing errors for file outlier.go: 'Error running tests: --- FAIL: TestQuartileOutliers_MildOutliers (0.00s)\n outlier_test.go:30: Expected mild out
liers to be detected\nFAIL\ncoverage: 99.0% of statements\nnext status 1\nFAIL\tgithub.com/montanaflynn/stats\t0.559s\n\n' Claude[Fix-outlier.go]: package stats

import "testing"

func TestQuartileOutliers_EmptyInput(t *testing.T) {
 _, err := QuartileOutliers(Float64Data{})
 if err != EmptyInputErr {
 t.Errorf("Expected EmptyInputErr for empty input, got %v", err)
 }
}

func TestQuartileOutliers_ExtremeOutliers(t *testing.T) {
 data := Float64Data{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, -100}
 result, err := QuartileOutliers(data)
 if err != nil {
 t.Errorf("Expected no error, got %v", err)
 }
 if len(result.Extreme) == 0 {
 t.Error("Expected extreme outliers to be detected")
 }
}

func TestQuartileOutliers_MildOutliers(t *testing.T) {
 data := Float64Data{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20}
 result, err := QuartileOutliers(data)
 if err != nil {
 t.Errorf("Expected no error, got %v", err)
 }
 if len(result.Mild) == 0 {
 t.Error("Expected mild outliers to be detected")
 }
}
[DONE Fix] outlier.go; success

Transitioned from FIXING to RUN_TEST
INFO:root:[RUN_TEST] : Running tests for selected file

< >

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ } Python

3.14.0 (my-agent)

Dependi

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

+

⌵

🗑

⋮

🔍

✕

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.1% | Files Left: 12

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[errors.go]: I'll help you write tests for the uncovered lines in errors.go. Let me first read the file to understand the code structure.

Claude[errors.go]: Now let me check if a test file already exists:

INFO:root:PreToolUse (ID: toolu_016Qikd1aGFYWWPeqVf2kRGX): Writing to test file: errors_test.go - ALLOWED

Claude[errors.go]: I've created the test file `errors\_test.go` with tests that cover the uncovered lines:

1. **TestStatsError_Error** - Tests the Error() method (lines 7-9)

2. **TestStatsError_String** - Tests the String() method (lines 11-13)

Both test functions use table-driven tests to verify that the methods return the correct error message string for various statsError instances.

[DONE] errors.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: errors.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.4% | Files Left: 11

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[cumulative_sum.go]: I'll read the file to understand the uncovered code and write appropriate tests.

Claude[cumulative_sum.go]: Now let me check if a test file exists and understand the types used:

Claude[cumulative_sum.go]: Let me check the data types and error definitions:

Claude[cumulative_sum.go]: Now I need to check what `EmptyInput` is (it's likely `ErrEmptyInput`):

Claude[cumulative_sum.go]: Let me check if EmptyInput is defined elsewhere:

Claude[cumulative_sum.go]: Perfect! Now I'll write the test file to cover the empty input case (lines 6-8):

INFO:root:PreToolUse (ID: toolu_01BshbTTT7MDoH6P4L48eZ83): Writing to test file: cumulative_sum_test.go - ALLOWED

[DONE] cumulative_sum.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: cumulative_sum.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

> old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.6% | Files Left: 10

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[max.go]: I'll help you write tests for the uncovered lines in max.go. First, let me read the file to understand the code structure.

Claude[max.go]: Now let me check if a test file already exists:

Claude[max.go]: Let me check what the Float64Data type and EmptyInputErr are:

Claude[max.go]: Now let me check if there's a reference to EmptyInputErr:

Claude[max.go]: Now I understand the code. The uncovered block is lines 11.22 to 13.3, which is the error case when the input is empty. I need to create a test file that tests this scenario.

INFO:root:PreToolUse (ID: toolu_011QsU3TmnsFBrch4uMKEUWc): Writing to test file: max_test.go - ALLOWED

[DONE] max.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: max.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

INFO:root:[RUN_TEST] Running tests for selected file | File: max.go | Fixes Left: 2

INFO:root:[RUN_TEST] : Transitioning to new state: FIXING

Transitioned from RUN_TEST to FIXING

INFO:root:[FIXING] : Fixing failing tests

INFO:root:Fixing errors for file max.go: 'Error running tests: FAIL\tgithub.com/montanaflynn/stats [build failed]\n\n# github.com/montanaflynn/stats [github.com/montanaflynn/stats.test]\n.n./max_test.go:4:2: "math" imported and not used\n'

[DONE Fix] max.go; success

Transitioned from FIXING to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.6% | Files Left: 9

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[median.go]: I'll help you write tests for the uncovered lines in median.go. Let me first read the file to understand the code structure.

Claude[median.go]: Now let me check if a test file already exists:

Claude[median.go]: Now let me check what types and functions are available to understand dependencies:

Claude[median.go]: Let me check the util.go file to understand the sortedCopy function:

><

main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

EXPLORES

OPEN EDITORS

main.py go-agent

agent_utils.py go-agent

models.py go-agent

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

> __pycache__

> .venv

.env

.python-version

agent_hooks.py

agent_utils.py

main.py

models.py

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Claude[median.go]: Let me check the util.go file to understand the sortedCopy function:

Claude[median.go]: Now let me check for EmptyInputErr:

Claude[median.go]: Now I'll create the test file. The uncovered block is lines 16.12 to 18.3, which is the empty input check. I need to test the case where an empty slice is passed to the Median function.

INFO:root:PreToolUse (ID: toolu_01CHvqowcBPPnW1FqVuXKGFz): Writing to test file: median_test.go - ALLOWED

[DONE] median.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: median.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.7% | Files Left: 8

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[min.go]: I'll help you write tests for the uncovered lines in min.go. Let me first read the file to understand the code structure.

Claude[min.go]: Now let me check if a test file already exists:

Claude[min.go]: Let me also check the package structure to understand the Float64Data type and EmptyInputErr:

Claude[min.go]: Now let me check for EmptyInputErr:

Claude[min.go]: Now I understand the code structure. The uncovered lines 12-14 correspond to the empty input error case. Let me write the test:

INFO:root:PreToolUse (ID: toolu_01Tn3H4qZF5VKQ47UckMWSyn): Writing to test file: /Users/rutu/stuff/vscode-workspaces/spectro/stats/min_test.go - ALLOWED

[DONE] min.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: min.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.7% | Files Left: 7

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk._internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[sigmoid.go]: I'll help you write tests for the uncovered lines in sigmoid.go. Let me first read the file to understand the code structure.

Claude[sigmoid.go]: Now let me check if a test file already exists:

Claude[sigmoid.go]: Let me check the types and errors used in the code:

Claude[sigmoid.go]: Now let me check the errors.go file to see the exact error name:

Claude[sigmoid.go]: Now I'll create the test file with a test that covers the uncovered lines (10.22 to 12.3), which is the empty input error case:

< main* ↻

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ }

Python

3.14.0 (my-agent)

Dependi

golang-test-agent

EXPLORER

OPEN EDITORS

main.py go-agent M

agent_utils.py go-agent M

models.py go-agent M

system_prompt.md go-agent

agent_hooks.py go-agent

GOLANG-TEST-AGENT

go-agent

__pycache__

.venv

.env

.python-version

agent_hooks.py

agent_utils.py M

main.py M

models.py M

pyproject.toml

README.md

system_prompt.md

uv.lock

old

.env

.gitignore

README.md

OUTLINE

TIMELINE

GO

PACKAGE OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

zsh - go-agent

Claude[sigmoid.go]: Now let me check if a test file already exists:

Claude[sigmoid.go]: Let me check the types and errors used in the code:

Claude[sigmoid.go]: Now let me check the errors.go file to see the exact error name:

Claude[sigmoid.go]: Now I'll create the test file with a test that covers the uncovered lines (10.22 to 12.3), which is the empty input error case:

INFO:root:PreToolUse (ID: toolu_01WhQzjKBSnaWS6GTVbF3hJw): Writing to test file: sigmoid_test.go - ALLOWED

[DONE] sigmoid.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: sigmoid.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 99.9% | Files Left: 6

INFO:root:[SELECT_FILE] : Transitioning to new state: GENERATE_TEST

Transitioned from SELECT_FILE to GENERATE_TEST

INFO:root:[GENERATE_TEST] : Generating tests for selected file

INFO:claude_agent_sdk.internal.transport.subprocess_cli:Using bundled Claude Code CLI: /Users/rutu/stuff/vscode-workspaces/spectro/golang-test-agent/go-agent/.venv/lib/python3.14/site-packages/claude_agent_sdk/_bundled/claude

Claude[softmax.go]: I'll read the softmax.go file to understand the uncovered code and write tests for it.

Claude[softmax.go]: Now let me check if a test file exists and examine the Float64Data type and EmptyInput error:

Claude[softmax.go]: Now I'll write the test file for softmax.go targeting the uncovered lines (9.22 to 11.3), which is the empty input check:

INFO:root:PreToolUse (ID: toolu_01CDafy3FmxHanbBkkedZxjH): Writing to test file: softmax_test.go - ALLOWED

[DONE] softmax.go; success

INFO:root:[GENERATE_TEST] Generating tests for selected file | File: softmax.go

INFO:root:[GENERATE_TEST] : Transitioning to new state: RUN_TEST

Transitioned from GENERATE_TEST to RUN_TEST

INFO:root:[RUN_TEST] : Running tests for selected file

Transitioned from RUN_TEST to SELECT_FILE

Selecting file for test generation

INFO:root:Target coverage 100.0% achieved with current coverage 100.0%. Terminating.

INFO:root:[SELECT_FILE] Selecting file for test generation | Target Coverage: 100.0% | Current Coverage: 100.0% | Files Left: 6

INFO:root:[SELECT_FILE] : Transitioning to new state: TERMINATE

Transitioned from SELECT_FILE to TERMINATE

=== AGENT TERMINATED ===

Final Coverage Achieved: 100.0%

Token Usage

Input Tokens: 405884

Output Tokens: 458309

Total Tokens: 864193

< main*

0 0

Live Share

Git Graph

-- NORMAL --

Not Committed Yet

Ln 135, Col 40

Spaces: 4

UTF-8

LF

{ } Python

3.14.0 (my-agent)

Dependi